# Práctica 5. Datos Anómalos

Análisis Exploratorio de Datos, Máster en Ciencia de Datos - UV

## Contents


**Introducción y objetivos** **1**

**Detección de inconsistencias en los datos** **2**

Ejercicio 1: Inspección Visual . . . . . 2

Ejercicio 2: Valores especiales . . . . . 2

Ejercicio 3: Valores perdidos y observaciones completas . . . . . 3

**Detección de outliers univariante** **3**

Ejercicio 4: Estadísticos básicos . . . . . 3

Ejercicio 5: Métodos de detección de outliers . . . . . 5

Ejercicio 6: Detección de outliers . . . . . 5

Ejercicio 7: Comparativa de los métodos de detección de outliers . . . . . 6

Ejercicio 8: Representación gráfica . . . . . 6

**Imputación de datos** **7**

Ejercicio 9: Imputación de datos perdidos con estadísticos básicos . . . . . 7

Ejercicio 10: Representación gráfica . . . . . 7


```
library(pacman)
library(dplyr)
library(magrittr)
packages = c("knitr", "ggplot2","tidyr","dplyr","readr")
pacman::p_load(char=packages)
```

## Introducción y objetivos

El objetivo de esta práctica es comprobar si los conjuntos de datos con los que vamos a trabajar son consistentes y, por tanto, pueden ser empleados para realizar un análisis sin errores. Las observaciones contenidas en los datos pueden presentar diferentes tipos de inconsistencias, como valores perdidos (missing values), valores especiales y valores anómalos (outliers).

Para evitar estas inconsistencias, estos datos deberán ser eliminados, corregidos o imputados. Siendo las etapas que pueden seguirse para obtener datos consistentes:

1. Detección de las inconsistencias.
2. Detectar el campo o campos que la generan y las dependencias entre ellos.
3. Corrección de errores.

Vamos a trabajar con el conjunto de datos *dirty_iris*, que ya conocemos, que contiene datos perdidos, valores especiales y anomalías de carácter numérico respecto a la distribución de los datos (`dirty_iris.csv`).

```
rm(list=ls())
dirty_iris <- read_csv("data/dirty_iris.csv")
```

## Detección de inconsistencias en los datos

Por lo general, no hay un convenio o una representación común para indicar un dato perdido o incluso determinados valores especiales. En los siguientes ejercicios vamos a analizar cómo detectar inconsistencias presentes en los datos.

### Ejercicio 1: Inspección Visual

Inspecciona visualmente los valores del conjunto de datos (`View`) y ordena de mayor a menor y de menor a mayor cada variable para descubrir las diferentes inconsistencias que presenta el conjunto `dirty_iris`. Comenta los valores que te llamen la atención.

```
# Medidas con valores faltantes, longitudes negativas y cero
#View(dirty_iris)
vars <- names(dirty_iris)


iris_Mm <- sapply(dirty_iris, function(x) sort(x, decreasing = TRUE)) #iris ordenado de Mayor a menor
iris_mM <- sapply(dirty_iris, function(x) sort(x, decreasing = FALSE))  #iris ordenado de menor a Mayor
```

Notamos que el número de observaciones por variable ahora ha cambiado, es decir, había `NA` en nuestros datos. - Cuando los hemos ordenador de mayor a menor vemos valores muy dispares entre observaciones consecutivas. Por ejemplo, en `Sepal.Length` vemos medidas de 73 hasta 7.9 y en `Petal.Width` vemos un valor `Inf`. - Cuando los hemos ordenador de menor a mayor vemos un valor negativo `Sepal.Width` y también valores de 0 en `Petal.Length`.

### Ejercicio 2: Valores especiales

Visualizar los datos puede ser muy revelador, pero en general, especialmente para conjuntos de datos grandes, realizaremos este análisis de forma automática.

Los valores especiales más comunes en R son:

- NA (*not available*): missing value o dato perdido (`is.na`)
- NULL (*empty set*): elemento nulo de longitud 0 (`is.null`)
- NaN (*not a number*): indeterminación con resultado desconocido (`is.nan`)
- Inf (*infinite*): valor numérico de valor infinito (`is.infinite`)

Nota: A parte de estas funciones, `is.numeric` comprueba si la variable es numérica; `is.finite` comprueba la existencia de cualquier valor especial en un vector numérico pero devuelve `FALSE` para variables de texto.

Reemplaza, en todas las columnas numéricas del data frame `dirty_iris`, los valores que sean especiales (usando `is.finite`), negativos, o iguales a cero, por `NA`. Nota: Puedes reemplazar los valores para cada variable del dataframe usando `mutate` con la función `ifelse`.

```
# DEBEMOS ELIMINAR TAMBIEN LOS VALROES IGUALES A CERO YA QUE SE TRATA DE LONGITUDES DE PARTES DE UNA FL

var_num <- sapply(dirty_iris,is.numeric)
iris_NA <- dirty_iris[var_num] %>%
  mutate(across(everything(), function(x) ifelse(!is.finite(x) | x <= 0, NA, x)))
iris_NA$Species <-dirty_iris$Species
```

### Ejercicio 3: Valores perdidos y observaciones completas

- Calcula el número y porcentaje de observaciones completas (muestras sin ningún dato perdido) empleando `complete.cases` (detecta todos los registros sin datos faltantes).
- Emplea después la función `na.omit` para quedarnos solo con las filas con datos completos en un nuevo data frame `clean_iris`.

```
numero_NA <-sum(complete.cases(iris_NA))
frac_NA <- numero_NA/length(complete.cases(iris_NA))
cat('Hay', numero_NA, 'NA\'s. Es decir,', frac_NA*100,'% de NA\'s')
```

```
Hay 92 NA's. Es decir, 61.33333 % de NA's
```

```
clean_iris <-na.omit(iris_NA)

cat('Pasamos de ', dim(dirty_iris)[1], 'filas y ', dim(dirty_iris)[2], 'columnas; a tener sin NA\'s', di
```

```
Pasamos de  150 filas y  5 columnas; a tener sin NA's 92 filas y  5 columnas
```

## Detección de outliers univariante

Un outlier es una observación (o conjunto de observaciones) que no parecen consistentes con el resto de valores de esa variable. Por lo general, esas observaciones divergirán del patrón de comportamiento del resto de datos, es decir, presentarán una desviación respecto de la distribución de los datos.

En esta sección, vamos a centrarnos en métodos de detección univariantes, así que, en los ejercicios, a modo de ejemplo, analizaremos solo la variable `Petal.Length`.

### Ejercicio 4: Estadísticos básicos

La presencia de outliers puede producir una gran influencia en caracterizaciones sencillas de los datos, como la media y la desviación típica.

Empleado la función `summarise y across`, calcula la media, la desviación típica, mediana, la *median absolute deviation from the median* (MADM) y el *interquartile range* (IQR) de las variables numéricas. Hazlo para el data frame con observaciones completas `clean_iris` (que no tiene NA pero tiene menos observaciones) y para el data frame con datos faltantes `dirty_iris` (que presenta muchos NA pero tiene todas las observaciones).

- Ten en cuenta que no se puede realizar un análisis estadístico con datos faltantes, pero en la mayoría de los casos se puede optar por no considerar dichos valores al realizar los cálculos (parámetro `na.rm=TRUE`).
- Para obtener el valor MADM, escribe una función `madm(x)` que sea robusta a la presencia de valores NA. Recuerda que el MADM se calcula como $1.4826 \cdot \text{median}(|x - \text{median}(x)|)$.
- Compara los resultados para los dos conjuntos en la variable `Petal.Length`.

```
madm <- function(x) {
  med_x <- median(x, na.rm = TRUE)
  1.4826 * median(abs(x - med_x), na.rm = TRUE)
}
resumen<-clean_iris[var_num]%>%summarise(across( .cols = everything(),
                                        .fns = list(
                                          media = ~ mean(.x, na.rm = TRUE), # Calcula la media
                                             sd = ~ sd(.x, na.rm = TRUE),
                                          mediana = ~ median(.x, na.rm = TRUE),
                                          IQR = ~ IQR(.x, na.rm = TRUE),
                                          MADM = ~madm(.x)
                                        ),
                                        .names = "{.col}_{.fn}"
                                        ))

resumen_clean <- pivot_longer(resumen, cols = everything(), names_to = c('variable', 'estadistico'), nam
resumen_clean %<>% pivot_wider( names_from = estadistico, values_from = value)
resumen_clean
```

```
# A tibble: 4 x 6
  variable     media    sd mediana   IQR  MADM
  <chr>        <dbl> <dbl>   <dbl> <dbl> <dbl>
1 Sepal.Length  6.37 4.57      5.8   1.3 1.04
2 Sepal.Width   3.33 2.84      3     0.5 0.371
3 Petal.Length  4.16 2.88      4.5   3.6 1.63
4 Petal.Width   1.22 0.765     1.4   1.6 0.890
```

Ahora con los datos sin limpiar

```
resumen<-dirty_iris[var_num]%>%summarise(across( .cols = everything(),
                                        .fns = list(
                                          media = ~ mean(.x, na.rm = TRUE), # Calcula la media
                                             sd = ~ sd(.x, na.rm = TRUE),
                                          mediana = ~ median(.x, na.rm = TRUE),
                                          IQR = ~ IQR(.x, na.rm = TRUE),
                                          MADM = ~madm(.x)
                                        ),
                                        .names = "{.col}_{.fn}"
                                        ))

resumen_dirty <- pivot_longer(resumen, cols = everything(), names_to = c('variable', 'estadistico'), nam
resumen_dirty %<>% pivot_wider( names_from = estadistico, values_from = value)
resumen_dirty
```

```
# A tibble: 4 x 6
  variable      media     sd mediana   IQR  MADM
```

```
  <chr>          <dbl>  <dbl>   <dbl> <dbl> <dbl>
1 Sepal.Length   6.56   6.80    5.75   1.3 0.964
2 Sepal.Width    3.39   3.32    3      0.5 0.297
3 Petal.Length   4.45   5.77    4.5    3.5 1.63
4 Petal.Width  Inf    NaN       1.3    1.5 1.04
```

La principal diferencia la vemos en `Petal.Width` donde usando los datos completamente en bruto tenemos un valor `Inf` y `NaN` en la media y la desviación típica, respectivamente. Este efecto es porque justamente en esa variable tenemos un valor `Inf` por lo tanto, contamina dichos estadísticos.

Los valores obtenidos en la media son parecidos, coincidiendo resultados entre 1 y 2 cifras significativas. La sd es la que más sufre al quitar los `NA` ya que es muy sensible al número de muestras. Por otro lado, la mediana y el IQR se mantienen muy estables al eliminar los valores faltantes. Por último, el MADM como depende de la media vemos grandes cambios entre los dos conjuntos.

## Ejercicio 5: Métodos de detección de outliers

Escribe una función que detecte los outliers de un vector $x$ para cada uno de los siguientes métodos de detección de outliers univariantes (todas las funciones tienen que ser robustas a la presencia de valores NA).

- **Regla 3 sigma** `reglasigma(x)`: Considera que los datos siguen una distribución gaussiana. Datos típicos caracterizados por la media y la dispersión por la varianza. Valores por encima de 3 desviaciones típicas se consideran outliers.
- **Identificador Hampel** `reglahampel(x)`: Utiliza estimadores robustos y no presupone que los datos siguen una distribución gaussiana. Datos típicos caracterizados por la mediana y la dispersión por la *median absolute deviation from the median* (MADM). Valores por encima de 3 MADM se consideran outliers.
- **Regla boxplot** `reglaboxplot(x)`: A partir de los datos se calcula su diagrama de caja (boxplot) y se definen como datos típicos superiores hasta el cuartil 75%, $x_s$; datos típicos inferiores hasta el cuartil 25%, $x_i$; y la dispersión se define como el rango intercuartílico ($IQR = x_s - x_i$). Valores por encima de $x_s + 1.5 \cdot IQR$ o por debajo de $x_i - 1.5 \cdot IQR$ se consideran outliers. Usa la función `quantile` para calcular los cuartiles.
- **Percentiles** `reglapercentil(x)`: Cualquier valor que esté fuera del rango del percentil 5% y 95% se puede considerar como atípico. Usa la función `quantile` para calcular los percentiles.

## Ejercicio 6: Detección de outliers

Utiliza la función `mutate` y `across` para identificar los outliers en la variable `Petal.Length` con todos los métodos de detección de outliers implementados. Genera una columna nueva para cada método. Nota: podría extenderse la aplicación a todas las variables numéricas usando también `mutate`, `across` y `where`.

```
outliers <- clean_iris %>% mutate(across(Petal.Length, list(reglasigma, reglahampel, reglaboxplot, regla
outliers
```

```
# A tibble: 92 x 9
   Sepal.Length Sepal.Width Petal.Length Petal.Width Species    Petal.Length_1 Petal.Length_2
          <dbl>       <dbl>        <dbl>       <dbl> <chr>      <lgl>          <lgl>
 1          6.4         3.2          4.5         1.5 versicolor FALSE          FALSE
 2          6.3         3.3          6           2.5 virginica  FALSE          FALSE
 3          5           3.4          1.6         0.4 setosa     FALSE          FALSE
```

```
 4          5.7         2.6          3.5         1   versicolor FALSE          FALSE
 5          5.9         3            5.1         1.8 virginica  FALSE          FALSE
 6          5.8         2.7          4.1         1   versicolor FALSE          FALSE
 7          4.8         3.1          1.6         0.2 setosa     FALSE          FALSE
 8          5           3.5          1.6         0.6 setosa     FALSE          FALSE
 9          6           2.7          5.1         1.6 versicolor FALSE          FALSE
10          6.8         2.8          4.8         1.4 versicolor FALSE          FALSE
# i 82 more rows
# i 2 more variables: Petal.Length_3 <lgl>, Petal.Length_4 <lgl>
```

## Ejercicio 7: Comparativa de los métodos de detección de outliers

En los métodos de detección de outliers analizados, examina los resultados comparando:

- Número de outliers detectados por método.
- Los valores etiquetados como outliers (ordenados de menor a mayor).

```r
sapply(outliers[-c(1:5)], function(x) sum(x == TRUE))
```

```
Petal.Length_1 Petal.Length_2 Petal.Length_3 Petal.Length_4
             2              2              2             10
```

```r
sort(outliers$Petal.Length[outliers[[6]] == TRUE])
```

```
[1] 14 23
```

```r
sort(outliers$Petal.Length[outliers[[7]] == TRUE])
```

```
[1] 14 23
```

```r
sort(outliers$Petal.Length[outliers[[8]] == TRUE])
```

```
[1] 14 23
```

```r
sort(outliers$Petal.Length[outliers[[9]] == TRUE])
```

```
 [1]  1.1  1.3  1.3  1.3  1.3  6.7  6.7  6.9 14.0 23.0
```

## Ejercicio 8: Representación gráfica

Representa gráficamente los datos de `Petal.Length` (distribución de puntos en el eje $y$) para todas las reglas de detección implementadas (repartidas en el eje $x$) mostrando los valores identificados como típicos y los identificados como outliers en diferente color. Usa la función `pivot_longer` para poder agrupar por reglas y colorear los outliers fácilmente.

Nota: usa en `geom_point` con la opción `position='jitter'` para mejorar la legibilidad del gráfico.

# Imputación de datos

Como se ha visto, con *missing data* nos referimos a una observación (o conjunto de observaciones) cuyo valor no es conocido.

Vamos a ver a continuación ejemplos de cómo imputar valores realistas a esas observaciones perdidas.

## Ejercicio 9: Imputación de datos perdidos con estadísticos básicos

La opción más sencilla es imputar con un valor estimado de los valores conocidos (por ejemplo, con la media, la mediana, o la moda).

Se puede realizar una imputación generalizada (usando todos los valores de la variable) o una imputación a partir de casos similares (la estimación se realiza teniendo en cuenta otra variable):

- Imputa los missing data en `Petal.Length` empleando la media. Nota: Puedes reemplazar los valores vacios usando `mutate` con `ifelse` creando una nueva variable `Petal.Length1`.
- Imputa los missing data en `Petal.Length` empleando en esta ocasión la media para cada tipo de `Species` (usar`group_by` y `mutate`) creando una nueva variable `Petal.Length2`.

## Ejercicio 10: Representación gráfica

Vamos a comparar ahora los dos métodos de imputación gráficamente eliminando los outliers para una mejor visualización.

- Recalcula las reglas de identificación de outliers (Ejercicio 6) para la nueva variable `Petal.Length2`.
- Representa los dos tipos de imputación empleados en un gráfico de dispersión (`Petal.Length1` vs. `Petal.Length2`) eliminando los outliers identificados con la regla `reglapercentil` y poniendo en diferente color los datos originales y los imputados.